Team No. 14

M1: Approach Document

Voice-Driven AI Incident Report Assistant (Qualcomm Problem Statement)

Target device: Qualcomm QIDK (Snapdragon 8 Elite, Hexagon NPU V79), fully on-device and offline

1 Problem Statement and Our Understanding

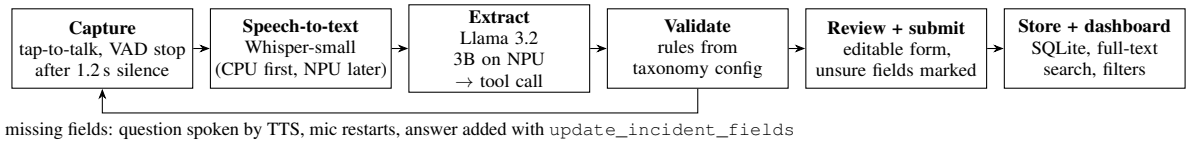
Reporting an incident usually means opening a portal and filling in many fields by hand. In practice, frontline staff such as technicians, guards and cleaners often do not report smaller incidents and near-misses at all, even though these are useful early warnings. The problem statement asks for a system where the user simply describes the incident by voice, for example *"There was a water leak near Conference Room 2 around 10 AM. Maintenance has already isolated the area."*, and the device produces a structured and validated report.

From the slides we identified five main requirements: (1) all AI models must run on the device using Snapdragon acceleration, with no cloud APIs; (2) the system should understand natural conversational speech rather than fixed commands; (3) speech recognition should be accurate; (4) incident details should be extracted accurately; and (5) the extracted details should be mapped to the correct report fields. The given flow also includes a check for mandatory fields that loops back to the user, function/tool calling to fill the form, a user review step before submission, and output both on screen and through text-to-speech.

2 Idea / Abstract

The user taps a single button and describes the incident in English or Hinglish. Whisper converts the speech to text on the device. Llama 3.2 3B Instruct, running on the Hexagon NPU, then classifies the incident and returns one JSON function call that fills the form. A rule-based validator (not a model) checks which mandatory fields are still missing for that incident type, and the app asks the user only for those fields by voice (at most two fields per question and at most three rounds). The user can also correct values by speaking. Once the report is complete, recommended next steps are shown from a fixed list of site SOPs, and the user reviews and submits it. Reports are stored in a searchable dashboard on the device. If critical keywords are detected (for example fire spreading, an unconscious person or a gas leak), an emergency banner with a one-tap dial option is shown immediately.

3 High-Level Approach, Pipeline and Architecture



Component	Design and reasoning
Function calling	The model only extracts information. The app defines and executes the functions: `fill_incident_form`, `update_incident_fields`, `submit_incident_report` and `trigger_emergency_banner`. The output is checked against a schema, enum values are normalised, and names, places and equipment are kept only if they actually appear in what the user said. Hinglish is converted to English in the same call, so no separate translation step is needed.
Incident model	10 general categories plus Other (Fire, Medical/Injury, Water leak, Electrical, Gas/Chemical, Equipment, Security, Structural, Environmental, Vehicle). The category is only used to choose which fields and questions apply. Common fields are type, description, location, date/time, severity, injuries, persons, equipment, actions taken, witnesses and recommended follow-up. Each category adds 2–3 mandatory fields (for Fire: fire status, evacuation status, fire brigade called).
Rule-based helpers	A time resolver converts phrases such as "around 10" or "aadha ghanta pehle" into a timestamp; if no time is given, the report time is used and marked for review. A fuzzy matcher maps locations to a site registry. Severity rules can raise the model's rating (for example, unconscious person → Critical) but never lower it.
On-device AI	Llama 3.2 3B Instruct as an AI Hub Genie bundle (w4a16, HTP V79); Qualcomm's published figures for this chip are about 28 tokens/s decode and 730 tokens/s prefill. We use greedy decoding for extraction, keep the models loaded, allow only one NPU session at a time, and retry temporary NPU errors. Whisper-small runs through whisper.cpp in the first version, with AI Hub NPU binaries as the planned upgrade. Our target is at most 8 s from the end of speech to a filled form.
App and runtime	Native Android app (Kotlin, Jetpack Compose, Room). All inference goes through one `InferenceEngine` interface: in-app JNI calls to Genie if the app can access the NPU (Plan A), or otherwise a local inference service started as the shell user on 127.0.0.1 (Plan B). In Plan A the app has no internet permission, and the demo will run in airplane mode with NPU timings shown on screen.

Worked example (input from the problem statement)

User says	"There was a water leak near Conference Room 2 around 10 AM. Maintenance has already isolated the area."
Model returns	`fill_incident_form{"incident_type": "WATER", "location": "Conference Room 2", "time": "around 10 AM", "actions_taken": ["Area isolated by maintenance"], "severity": "MEDIUM", "description": "Water leak near Conference Room 2; area isolated.", "details": {"leak_status": "CONTROLLED"}}`
Code then	Resolves the time to 10:00 today (approximate), matches "Conference Room 2" in the site registry, and the validator finds that *injuries* and *leak source* are still missing.
App asks (voice)	"Was anyone hurt, and where was the water coming from?" The user replies "No one, it was the AC drain", which gives `update_incident_fields{"injuries": "NO", "details": {"leak_source": "AC drain"}}`. The form is now complete, SOP steps are shown, and the user reviews and submits.

How this differs from a basic speech-to-form app: the app asks only for the mandatory fields that are missing and accepts corrections by voice; times, places and severity limits are handled by code rather than the model, which also prevents invented details; the language model runs on the NPU with live timings in airplane mode; and accuracy will be measured against a baseline instead of only being claimed.

4 Planned Testing Methodology

Test data

- **About 150 text cases** generated from templates across all 10 categories, covering complete reports, missing mandatory fields, self-corrections ("at 10, no, 11"), filler words, relative times, Hinglish and incidents with more than one category. Each case has the expected value (or null) for every field.
- **30 recorded audio clips** in English and Hinglish, with different speakers and accents and some background noise, recorded on the QIDK microphone, each with a reference transcript and expected form.
- The dev and held-out split is fixed before prompt tuning, and final results are reported only on the held-out set.

Baseline: a rules and keyword based extractor (keyword lists, regex for times, location registry, no LLM), so the improvement from the model can be measured directly.

Criterion	Metric
Speech recognition	WER (English) and CER (Hinglish) on the 30 clips
Extraction	Incident-type accuracy; per-field precision, recall and F1
Field mapping	Mapping accuracy (correct value in the correct field), reported separately
Reliability	Valid tool-call rate; rate of fields filled when the expected value is null
Dialogue	Whether follow-ups ask for exactly the missing fields; turns to completion
On-device	p50/p90 latency per stage, tokens/s, NPU temperature, airplane-mode run

We will also write unit tests for the validator, time resolver, location matcher and tool-call parser, build an on-device evaluation runner that passes every test case through the real NPU pipeline one at a time, and run an end-to-end test of 20 consecutive spoken reports on the board before the demo.

5 Status and Plan

- **Done so far:** requirement analysis and a full specification; board setup (clock fixed, storage cleared, runtime prepared); the Llama 3.2 3B Genie export for V79 is running on two machines in parallel; the QAIRT 2.50 NPU runtime is downloading.
- **Planned for M2:** a first end-to-end run on the board (speech → transcript → tool-call JSON → validated form), an app skeleton with the review screen and dashboard, and first evaluation results.
- **Risks and fallbacks:** if the export is delayed we will use Llama 3.2 1B, and then llama.cpp on the CPU; if Whisper on the NPU is not ready we will use whisper.cpp on the CPU; if the app cannot access the NPU we will use the shell-user inference service. Future scope, outside the offline demo: LLM-tailored recommendations, conversational queries over the incident database, a Hinglish fine-tuned Whisper and optional sync.